SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

Course Code:	DSA05	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	Tool 2 – Schema Analysis and Viva Voce
Weightage:		**Total Marks:**	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)			
Student Name:	Nivya Sri T	**Reg. No.:**	192424131
Section / Batch:	2024	**Date of Submission:**	05-08-26
Faculty In charge:	K. Veena Devi	**Project Mode:**	Individual Submission

Code of Conduct

I ___Nivya Sri T____ (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____

Identification of Entities and Attributes (4 Marks)	Understanding of Relationships (4 Marks)	Analysis of Keys and Constraints (4 Marks)	Detection of Design Issues (4 Marks)	Viva Voce (4 Marks)	Total (20 Marks)

1. University Course Registration System

A university database contains the following tables:

- Student(StudentID, Name, DepartmentID)
- Department(DepartmentID, DepartmentName)
- Course(CourseID, CourseName, FacultyID)
- Faculty(FacultyID, FacultyName)
- Enrollment(StudentID, CourseID, Semester)

A student can enroll in multiple courses, and each course can have multiple students.

Viva Questions

Q1. Why is the Enrollment table necessary instead of storing CourseID directly in the Student table?

Because Student–Course is a many-to-many relationship, you need a separate Enrollment table. A single CourseID in the Student table cannot store multiple courses, violates normalization, and cannot hold extra details like semester or grades.

Q2. If the university wants to track grades for each enrolled course, how would you modify the schema?

Add a Grade attribute to the Enrollment table:

Enrollment(StudentID, CourseID, Semester, Grade)

This stores grades per student per course without changing other tables.

2. Hospital Management System

A hospital database contains:

- Patient(PatientID, Name, Age)
- Doctor(DoctorID, DoctorName, Specialization)
- Appointment(AppointmentID, PatientID, DoctorID, Date)
- Prescription(PrescriptionID, AppointmentID, Medicine)

Multiple patients can consult the same doctor.

Viva Questions

Q1. What problems might occur if prescription details are stored directly in the Appointment table?

Storing prescription details directly in the Appointment table causes problems because:

- An appointment may require multiple medicines, which violates normalization.
- It makes the Appointment table large and repetitive.
- You cannot store extra prescription-specific details (dosage, duration, frequency).

That's why a separate Prescription table is needed.

Q2. How would the schema change if a prescription contains multiple medicines?

If a prescription contains multiple medicines, modify the schema by creating a separate table:

Prescription(PrescriptionID, AppointmentID)

PrescriptionMedicine(PrescriptionID, Medicine, Dosage, Duration)

This allows storing any number of medicines for each prescription cleanly.

3. Online Shopping System

Database tables include:

- Customer(CustomerID, Name)
- Product(ProductID, ProductName, Price)
- Order(OrderID, CustomerID, OrderDate)
- OrderDetails(OrderID, ProductID, Quantity)

Each order may contain multiple products.

Viva Questions

Q1. Why is the OrderDetails table required in this schema?

The OrderDetails table is required because an order can contain multiple products. It correctly represents the one-to-many relationship between Order and Products and allows storing extra details like Quantity, Price at purchase, etc.

Q2. What would happen if ProductID were stored directly in the Order table?

If ProductID were stored directly in the Order table:

- You could store only one product per order, which is incorrect.
- Storing multiple ProductIDs (comma-separated) breaks normalization.
- You cannot store quantity or other product-specific details.

Hence, OrderDetails is essential for multi-product orders.

4. Library Management System

Database tables:

- Member(MemberID, Name)
- Book(BookID, Title, Author)
- Issue(MemberID, BookID, IssueDate, ReturnDate)

Members can borrow multiple books.

Viva Questions

Q1. Explain the relationship between Member and Book through the Issue table.

The Issue table represents a many-to-many relationship between Member and Book.

- A member can borrow multiple books.

A book can be borrowed by multiple members (at different times).

- The Issue table stores each borrowing event with its IssueDate and ReturnDate.

Q2. How would you modify the schema to track overdue fines?

To track overdue fines, add a Fine field to the Issue table:

Issue(MemberID, BookID, IssueDate, ReturnDate, Fine)

Or, if detailed fine history is needed, create a separate table:

Fine(IssueID, Amount, PaidStatus)

5. Banking System

Tables:

- Customer(CustomerID, Name)
- Account(AccountNo, CustomerID, Balance)
- Transaction(TransactionID, AccountNo, Amount, Type)

One customer can have multiple accounts.

Viva Questions

Q1. Why should transactions be stored separately from account details?

Transactions must be stored separately because:

- An account has many transactions, so storing them inside the Account table breaks normalization.
- The Account table should store only current balance, not the entire transaction history.

- Separate storage allows tracking deposits, withdrawals, timestamps, and types clearly.

Q2. How would you redesign the schema to support joint accounts?

To support joint accounts, allow multiple customers to be linked to one account by creating a bridge table:

AccountHolder(AccountNo, CustomerID)

This represents a many-to-many relationship between Customer and Account.

6. Employee Payroll System

Tables:

- Employee(EmployeeID, Name, DepartmentID)
- Department(DepartmentID, DepartmentName)
- Payroll(PayrollID, EmployeeID, Salary, Month)

Employees belong to departments and receive monthly salaries.

Viva Questions

Q1. Why is Department information separated into a different table?

Department information is kept in a separate Department table to avoid duplication. If many employees belong to the same department, storing department details inside each Employee record would repeat the same data multiple times and violate normalization.

Q2. What redundancy issues would arise if department details were stored in Employee records?

If department details were stored inside Employee records:

- The same department name would be **repeated for every employee**, causing redundancy.
- Updating a department name would require changing it in **many rows**, increasing chances of inconsistency.

Separating Department ensures clean, consistent, and non-redundant data.

7. Hotel Reservation System

Tables:

- Customer(CustomerID, Name)
- Room(RoomID, RoomType)
- Reservation(ReservationID, CustomerID, RoomID, CheckIn, CheckOut)

Rooms can be booked by different customers at different times.

Viva Questions

Q1. Why should reservation details be maintained separately from room details?

Reservation details must be stored separately because:

- A room can be booked many times by different customers on different dates.
- Storing reservation info inside the Room table would cause repetition and violate normalization.
- Room table should only describe the room, not its booking history.

Q2. How would you prevent double booking of rooms using the schema?

To prevent double booking, enforce a rule that a room cannot have overlapping reservations.
This can be done by:

- Adding a unique constraint on (RoomID, CheckIn, CheckOut) ranges, or
- Validating during insertion that no existing reservation overlaps with the new one.

This ensures each room is booked only once for any given time period.

8. Food Delivery Application

Tables:

- Customer(CustomerID, Name)
- Restaurant(RestaurantID, Name)
- Order(OrderID, CustomerID, RestaurantID)
- DeliveryAgent(AgentID, Name)
- Delivery(OrderID, AgentID)

Orders are delivered by delivery agents.

Viva Questions

Q1. Why is Delivery maintained as a separate table?

Delivery is kept as a separate table because:

- An order is delivered by an agent, which is a separate relationship from customer or restaurant.
- It allows storing delivery-specific details (agent, time, status) without cluttering the Order table.
- Keeps the schema normalized and flexible.

Q2. How would the schema change if multiple delivery agents could handle a single order?

If multiple delivery agents can handle a single order, make the relationship many-to-many by creating a bridge table:

OrderAgent(OrderID, AgentID)

This allows assigning any number of agents to one order cleanly.

9. Learning Management System (LMS)

Tables:

- Student(StudentID, Name)
- Course(CourseID, CourseName)
- Enrollment(StudentID, CourseID)
- Assignment(AssignmentID, CourseID)

Students can enroll in many courses.

Viva Questions

Q1. Why is Enrollment considered a bridge table?

Enrollment is a bridge table because it connects Students and Courses in a many-to-many relationship. A student can take many courses, and a course can have many students — the Enrollment table links them cleanly.

Q2. How would you modify the schema to record assignment grades?

To record assignment grades, add a new table that links students, assignments, and grades:

AssignmentGrade(StudentID, AssignmentID, Grade)

This stores grades for every assignment submitted by each student.

10. Bus Ticket Reservation System

Tables:

- Passenger(PassengerID, Name)
- Bus(BusID, Route)
- Booking(BookingID, PassengerID, BusID, TravelDate)

Passengers can make multiple bookings.

Viva Questions

Q1. Why should booking details be stored separately from passenger information?

Booking details must be stored separately because:

- A passenger can make many bookings, so storing booking info inside the Passenger table breaks normalization.

- Passenger table should store only personal details, not travel history.
- Booking requires extra fields like BusID and TravelDate that don’t belong to Passenger.

Q2. If a booking contains multiple passengers, how would you redesign the schema?

If a booking contains multiple passengers, redesign the schema using a bridge table:

BookingPassenger(BookingID, PassengerID)

This allows linking any number of passengers to a single booking cleanly.